⚡ Apache Spark — architektura, RDD i podstawy PySpark

Jak Spark myśli o danych, zanim w ogóle zrobi cokolwiek

Python w Zastosowaniach Big Data | Wykład 4

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Przypomnienie — dlaczego w ogóle Spark? (Hadoop był wolny 🐌)
2. Architektura Sparka — driver, executory, cluster manager
3. RDD — Resilient Distributed Dataset, czyli DNA Sparka
4. Transformacje vs akcje — leniwość (lazy evaluation) i DAG
5. Najczęstsze operacje RDD w praktyce
6. Partycjonowanie i shuffle — gdzie się chowa wydajność
7. Pułapki, których należy unikać

> 🎯 **Po tym wykładzie:** Uruchomisz `SparkSession` w Colabie, zbudujesz RDD i wykonasz na nim transformacje oraz akcje — świadomie rozumiejąc, co dzieje się "pod maską".

💡 Dlaczego to ważne?

Pamiętacie z Wykładu 3: **MapReduce zapisuje wyniki każdego kroku na dysk**.

Efekt? Zadanie iteracyjne (np. 10 rund algorytmu) = 10× odczyt/zapis na dysk. Boli. 😬

Apache Spark trzyma dane **w pamięci RAM** między krokami.

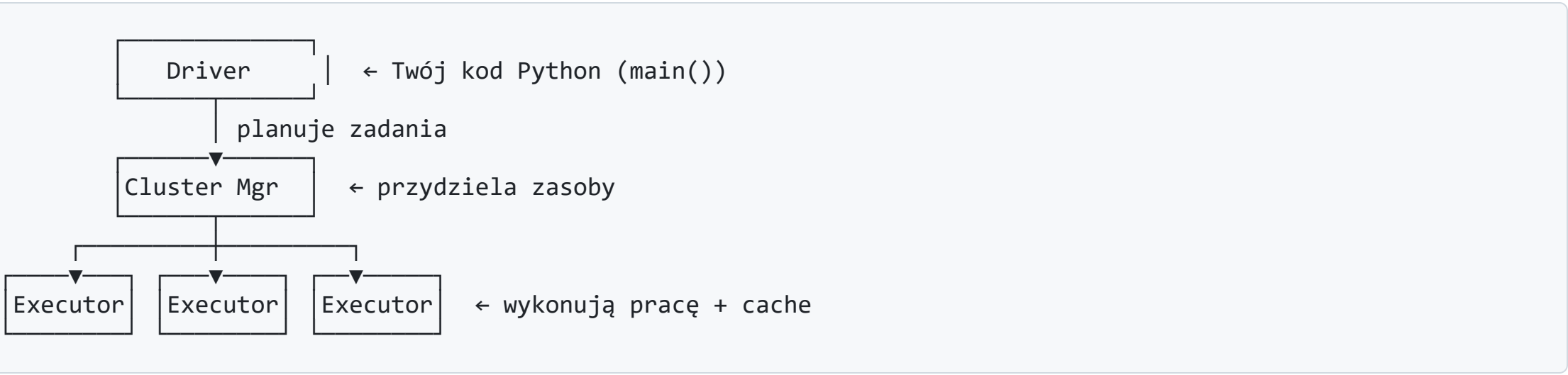
Operacja	Hadoop MapReduce	Apache Spark
Sortowanie 100 TB	~72 min (dysk)	~23 min (RAM + dysk)
Regresja logistyczna (10 iteracji)	~110 s/iterację	~0.9 s/iterację

> To nie jest magia — to architektura. Zobaczmy, jak ona działa. 🔍

🏗️ Architektura Sparka — kto tu rządzi?

Aplikacja Spark składa się z trzech głównych aktorów:

- **Driver** — "mózg" aplikacji. Tworzy `SparkContext` / `SparkSession`, planuje pracę, zbiera wyniki
- **Cluster Manager** — przydziela zasoby (standalone, YARN, Kubernetes, lub `local[*]` w Colabie)
- **Executory** — procesy robocze na węzłach klastra, które faktycznie wykonują zadania i trzymają dane w pamięci



🤔 Think about it

Pytanie: Gdy uruchamiasz Sparka w Google Colab z `master("local[*]")`, ile masz executorów?

🧬 RDD — Resilient Distributed Dataset

RDD to podstawowa struktura danych Sparka — kolekcja obiektów rozproszona na wielu węzłach.

Nazwa mówi wszystko:

Litera	Znaczenie	Co to daje?
Resilient	Odporny na awarie	Spark odtworzy utracone dane z **lineage** (historii transformacji)
Distributed	Rozproszony	Dane podzielone na partycje, na wielu maszynach
Dataset	Zbiór danych	Kolekcja elementów — dowolnych obiektów Python

RDD są **niemutowalne (immutable)** — każda transformacja tworzy **nowe** RDD, nigdy nie modyfikuje starego.

> 💭 Myśl o RDD jak o liście Pythona, tylko rozrzuconej na 50 komputerach naraz — i "tylko do odczytu" po utworzeniu.

⚙️ SparkSession i SparkContext w Colabie

```
# @title 🔧 Setup — instalacja i inicjalizacja Sparka
!pip install -q pyspark

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("Wyklad4-RDD-Podstawy")
    .master("local[*]")          # 👈 tryb lokalny — wszystkie rdzenie CPU
    .getOrCreate()
)

sc = spark.sparkContext          # 👈 SparkContext — "wejście" do świata RDD
print(sc.version)                # np. 3.5.1
print(f"Liczba partycji domyślnie: {sc.defaultParallelism}")
```

`SparkSession` to punkt wejścia od Spark 2.0 — obejmuje SQL, DataFrame i (przez `.sparkContext`) klasyczne RDD.

🏗️ Tworzenie pierwszego RDD

```
# Z listy Pythona (parallelize)
liczby = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(liczby.collect())  # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Z pliku tekstowego (każda linia = jeden element RDD)
# linie = sc.textFile("access_log.txt")

print(f"Liczba partycji: {liczby.getNumPartitions()}")
```

RDD można stworzyć na dwa główne sposoby: `sc.parallelize()` (z kolekcji w pamięci driver'a) lub `sc.textFile()` / inne źródła zewnętrzne.

Transformacje vs akcje — kluczowy podział

	Transformacje	Akcje
Co robią	Definiują *co* ma się stać z danymi	Faktycznie *uruchamiają* obliczenia
Kiedy wykonują się	Nigdy od razu (leniwe!)	Natychmiast
Zwracają	Nowe RDD	Wynik (liczba, lista, plik)
Przykłady	`map`, `filter`, `flatMap`, `reduceByKey`	`collect`, `count`, `take`, `reduce`, `saveAsTextFile`

```
# To NIC nie liczy — tylko buduje plan (DAG)
podwojone = liczby.map(lambda x: x * 2)

# Dopiero TERAZ Spark faktycznie wykonuje pracę
wynik = podwojone.collect()  # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

😴 Lazy evaluation — dlaczego Spark jest "leniwy"?

Spark **nie wykonuje** transformacji od razu. Zamiast tego buduje **graf skierowany acykliczny (DAG)** operacji.

Zalety leniwości:

- Spark może **zoptymalizować cały łańcuch** operacji naraz (np. połączyć kilka `map` w jeden przebieg)
- Nie marnuje pamięci na wyniki pośrednie, których nigdy nie użyjesz
- Pozwala odtworzyć dane po awarii — Spark zna cały "przepis" (lineage), a nie tylko dane

```
# Budujemy łańcuch — NIC się jeszcze nie dzieje
wynik = (
    sc.parallelize(range(1, 1_000_001))
    .filter(lambda x: x % 2 == 0)     # transformacja
    .map(lambda x: x ** 2)            # transformacja
    .filter(lambda x: x < 1000)       # transformacja
)
print(wynik.take(3))  # 👈 DOPIERO TERAZ Spark liczy — i to tylko tyle, ile potrzeba!
```

🗺️ Lineage i DAG — pamięć odporna na awarie

Każde RDD "pamięta", z jakiego RDD powstało i jaką transformacją.

```
RDD_0 (parallelize)
   │ .filter(parzyste)
RDD_1
   │ .map(kwadrat)
RDD_2
   │ .filter(< 1000)
RDD_3  ← jeśli partycja tego RDD zginie (awaria węzła),
         Spark odtworzy ją, przeliczając od RDD_0!
```

To właśnie ta historia transformacji (**lineage**) to sekret litery **R** w RDD — Resilient.

> ⚠️ Cena za tę odporność: transformacje na RDD są **niemutowalne** — nie da się "podmienić" elementu w miejscu.

🛠️ Najczęstsze operacje RDD — `map`

```
logi = sc.parallelize([
    "192.168.1.1 GET /home 200",
    "10.0.0.5 GET /api/users 404",
    "172.16.0.3 POST /login 500",
])

# map: 1 wejście -> 1 wyjście (transformacja elementu)
endpointy = logi.map(lambda linia: linia.split(" ")[2])
print(endpointy.collect())
# ['/home', '/api/users', '/login']
```

`map()` stosuje funkcję do **każdego** elementu i zwraca RDD tej samej długości.

🛠️ Najczęstsze operacje RDD — `filter`

```
# filter: zostawia tylko elementy spełniające warunek
bledy = logi.filter(lambda linia: linia.split(" ")[-1].startswith(("4", "5")))
print(bledy.collect())
# ['10.0.0.5 GET /api/users 404', '172.16.0.3 POST /login 500']
```

`filter()` **nie zmienia** liczby ani struktury elementów — usuwa te, dla których funkcja zwraca `False`.

🛠️ Najczęstsze operacje RDD — `flatMap`

```
zdania = sc.parallelize(["Spark jest szybki", "RDD jest odporny na awarie"])

# map zwróciłby listę list: [['Spark', 'jest', 'szybki'], [...]]
# flatMap "spłaszcza" wynik do jednej płaskiej listy słów
slowa = zdania.flatMap(lambda zdanie: zdanie.split(" "))
print(slowa.collect())
# ['Spark', 'jest', 'szybki', 'RDD', 'jest', 'odporny', 'na', 'awarie']
```

> 💡 **Zasada kciuka:** Użyj `flatMap`, gdy Twoja funkcja zwraca **listę** dla każdego elementu, a chcesz płaski wynik.

🛠️ Najczęstsze operacje RDD — `reduceByKey` i `groupByKey`

```
pary = sc.parallelize([("/home", 1), ("/login", 1), ("/home", 1), ("/api", 1)])

# reduceByKey: agreguje wartości dla tego samego klucza — EFEKTYWNIE
liczniki = pary.reduceByKey(lambda a, b: a + b)
print(liczniki.collect())
# [('/home', 2), ('/login', 1), ('/api', 1)]
```

```
# groupByKey: grupuje wszystkie wartości pod kluczem — potencjalnie DUŻO shuffle
grupy = pary.groupByKey().mapValues(list)
print(grupy.collect())
# [('/home', [1, 1]), ('/login', [1]), ('/api', [1])]
```

> ⚠️ `reduceByKey` redukuje dane **lokalnie na każdym executorze przed shuffle** — `groupByKey` przesyła wszystko przez sieć. Zawsze preferuj `reduceByKey`!

🧩 Partycjonowanie — jak Spark dzieli dane

RDD jest podzielone na **partycje** — niezależne kawałki danych, przetwarzane równolegle przez różne executory.

```
dane = sc.parallelize(range(1, 101), numSlices=4)  # 👈 wymuszamy 4 partycje
print(dane.getNumPartitions())  # 4

# Zobacz, ile elementów trafiło do każdej partycji
print(dane.glom().map(len).collect())  # np. [25, 25, 25, 25]
```

Za mało partycji	Za dużo partycji
Executory bezczynne, słabe zrównoleglenie	Narzut na zarządzanie zadaniami (overhead)

> 🎯 Reguła kciuka: 2–4 partycje na rdzeń CPU w klastrze.

🔀 Shuffle — najdroższa operacja w Sparku

Shuffle = przetasowanie danych między partycjami/węzłami przez sieć. Kosztowne: dysk + sieć + serializacja.

Operacje, które **zawsze** powodują shuffle:

- `groupByKey`, `reduceByKey`, `sortByKey`
- `join`, `distinct`, `repartition`

```
# 🚨 Shuffle: dane muszą "spotkać się" na tym samym executorze wg klucza
wynik = pary.reduceByKey(lambda a, b: a + b)
```

> 💡 Nie da się uniknąć shuffle przy agregacji po kluczu — ale można go **zminimalizować**, redukując dane **przed** shuffle (stąd wyższość `reduceByKey` nad `groupByKey`).

⚠️ Pułapka #1 — `collect()` na ogromnych danych

```
# 😱 NIE RÓB TEGO na RDD z milionami rekordów!
wszystko = ogromne_rdd.collect()  # sprowadza WSZYSTKO do pamięci drivera -> OOM!
```

Rozwiązanie: Używaj `take(n)`, `takeSample()`, `count()` albo zapisuj wynik na dysk (`saveAsTextFile()`).

```
podglad = ogromne_rdd.take(10)          # ✅ bezpieczne – tylko 10 elementów
liczba = ogromne_rdd.count()            # ✅ bezpieczne – zwraca liczbę
```

⚠️ Pułapka #2 — niepotrzebne `groupByKey`

```
# 😬 Wolniej: przesyła WSZYSTKIE wartości przez sieć
suma = dane.groupByKey().mapValues(sum)

# 🚀 Szybciej: redukuje lokalnie PRZED shuffle
suma = dane.reduceByKey(lambda a, b: a + b)
```

⚠️ Pułapka #3 — brak `cache()` przy wielokrotnym użyciu

```
rdd = sc.parallelize(range(1_000_000)).map(lambda x: x ** 2)
rdd.cache()  # 👈 zapisz wynik w pamięci — bo użyjemy go kilka razy!

print(rdd.count())     # pierwsze użycie: liczy I zapisuje w cache
print(rdd.take(5))     # drugie użycie: czyta z cache, NIE liczy od nowa
```

Bez `cache()` Spark **przelicza cały lineage od nowa** za każdym razem, gdy wywołujesz akcję!

📝 Kluczowe wnioski

1. ✅ Spark = driver + executory + cluster manager, dane trzymane w RAM
2. ✅ RDD to niemutowalna, rozproszona kolekcja z lineage (odporność na awarie)
3. ✅ Transformacje są leniwe — liczą się dopiero przy akcji
4. ✅ `reduceByKey` > `groupByKey` — mniej danych przez sieć
5. ✅ `cache()` ratuje wydajność przy wielokrotnym użyciu tego samego RDD

🔮 Co dalej?

Wykład 5 (03.11.2026): Spark SQL i DataFrames — czyli jak dodać do RDD schemat, optymalizator Catalyst i… zwykły SQL 🎉

Ćwiczenia dziś: Analiza ~50 000 syntetycznych logów serwera WWW przy użyciu RDD — parsowanie linii, `map` / `filter` / `reduceByKey`, wykrywanie botów i najwolniejszych requestów.

❓ Pytania?

Dziękuję za uwagę! 🎉

Zaraz przechodzimy do ćwiczeń praktycznych — otwórzcie notebook `cw-04-pyspark-rdd-logi-serwera.ipynb` 💻